



where  $\dot{\mathbf{x}} \in \mathbb{R}^6$  is the task space velocity (translational and angular velocities) of the end-effector and the time derivative of the end-effector pose  $\mathbf{x} \in \mathbb{R}^6$  :

$$\mathbf{x} = [\mathbf{x}_\nu^T \quad \mathbf{x}_\omega^T]^T \quad (3)$$

The  $\mathbf{J}$  matrix can be computed using forward kinematics and written using two matrices :

$$\mathbf{J} = \frac{\partial \mathbf{x}}{\partial \mathbf{q}} = [\mathbf{J}_\nu^T \quad \mathbf{J}_\omega^T]^T \quad (4)$$

where  $\mathbf{J}_\nu \in \mathbb{R}^{3 \times n}$  is its translational part and  $\mathbf{J}_\omega \in \mathbb{R}^{3 \times n}$  is its angular part.

### III. IMPACT MODELING

In this section, we propose a mathematical model of the impact between the hammer tip and the nail. This part is heavily inspired by Stronge [14] and his particle impact model.

#### A. Common tangent plane of collision

From Stronge [14], we define the common tangent plane of collision as the plane passing through the contact point at the instant of collision that is tangent to both surfaces involved in the collision and the normal unit vector to the collision  $\mathbf{n} \in \mathbb{R}^3$  as a vector orthogonal to that plane, see the schematic on Fig. 1. Let  $\mathbf{R}_{w \rightarrow \text{nail}} \in SO(3)$  be the rotation matrix of the nail with respect to the world frame  $w$ . Then the normal vector expressed in the world frame  $\mathbf{n}_w$  is :

$$\mathbf{n}_w = \mathbf{R}_{w \rightarrow \text{nail}}^T \mathbf{n} \quad (5)$$

#### B. Effective mass of the robot

Let us define  $m_{e,n}(\mathbf{q}) \in \mathbb{R}^+$  (the "e" stands for "effective"), the configuration dependant Effective Mass of the Robot (EMR), as :

$$m_{e,n}(\mathbf{q}) \triangleq (\mathbf{n}^T \mathbf{\Lambda}(\mathbf{q}) \mathbf{n})^{-1} \quad (6)$$

where  $\mathbf{\Lambda} \in \mathbb{R}^{3 \times 3}$  is the mobility matrix [8] defined as :

$$\mathbf{\Lambda}(\mathbf{q}) \triangleq \mathbf{J}_\nu(\mathbf{q}) \mathbf{M}^{-1}(\mathbf{q}) \mathbf{J}_\nu^T(\mathbf{q}) \quad (7)$$

The EMR represents the equivalent mass of the robot along the direction of  $\mathbf{n}$  if we were to replace the robot by a particle.

#### C. Choice of the model

We have three main ways to solve this impact problem: i) a force-based model, ii) an energy-based model and iii) a momentum model.

1) *Force-based model*: The force-based model relies on Newton's second law :

$$\mathbf{F}^h = m_{e,n} \frac{d\dot{\mathbf{x}}_\nu}{dt} \quad (8)$$

where  $\dot{\mathbf{x}}_\nu \in \mathbb{R}^3$  is the translational part of  $\dot{\mathbf{x}}$  defined in (2),  $\mathbf{F}^h \in \mathbb{R}^3$  is the force applied to the nail by the hammer (the superscript "h" stands for "hammer") and  $m_{e,n}$  is the Effective Mass of the Robot (EMR) at the point of impact in the normal of the impact plane defined in (6). The

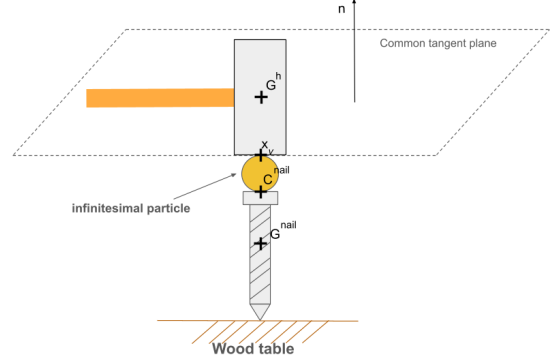


Fig. 1. Schematic of the collision between the hammer tip and the nail modeled as a particle impact

problem with this model is that the discontinuous velocity of the hammer tip at the instant of collision,  $t_c$ , as the time derivative at this point in time does not exist. We can not use a force-based model.

2) *Energy-based model*: An energy-based model could be used to overcome the problem of the discontinuity of the hammer tip velocity. However, during impact, the kinetic energy of the hammer tip is not fully transmitted to the nail: some of the energy is converted into heat, sound and deformation energy, which are difficult to estimate. Thus, although the model is still usable, we would either have to estimate the losses or neglect them making the model less precise.

3) *Momentum-based model*: The momentum based model does not have the problems of the force-based model and the energy-based model. With momentum, the microscopic interactions between the hammer tip and the nail are taken into account through a scalar number  $e_*$  called *coefficient of restitution*, which measures the elasticity of the collision. Furthermore, another advantage of using this model is its ability to deal with discontinuous velocities, which occur when hitting a nail with a hammer. As such, we use the momentum model in the following part of the paper.

More precisely, we use a model based on particle impact. A schematic depicting the problem is proposed in Fig. 1. In this model, an infinitesimal particle (i.e. with negligible size) models the local deformations of the hammer tip and the nail.

#### D. Direct impact

Let  $\mathbf{x}_\nu(t)$  and  $\mathbf{c}^{\text{nail}}(t) \in \mathbb{R}^3$  be the contact points of the hammer tip and the nail expressed in the world frame  $w$ , respectively, as illustrated by Fig. 1. At the instant of contact  $t_c$ , these points collide and thus share the same coordinates (neglecting the radius of the particle, see Fig. 1) :

$$\mathbf{x}_\nu(t_c) = \mathbf{c}^{\text{nail}}(t_c) \quad (9)$$

The nail is stationary before impact, i.e. from the initial time  $t_i$  to the instant of contact  $t_c$  its velocity is  $\mathbf{0}$ :

$$\dot{\mathbf{c}}^{nail}(t) = \mathbf{0} \quad \forall t \in [t_i, t_c] \quad (10)$$

The velocity and acceleration of the hammer tip is given by the trajectory that will be introduced in Section V-C.

A direct impact is an impact that occurs when the relative approaching velocity of the colliding bodies is parallel to  $\mathbf{n}$  [14]. Our goal with the hammering problem is to achieve a direct impact. We will assume that it is indeed the case, thus there will be no tangential sliding during impact.

#### E. Projected momentum of the hammer tip

To quantify an impact, we define the normal impulse  $i \in \mathbb{R}$  given by the hammer tip to the nail over an infinitesimal duration  $\delta t$  to be:

$$i \triangleq \int_{t_c}^{t_c + \delta t} \mathbf{n}^T \mathbf{F}^h dt \quad (11)$$

To maximize the impact, we want to maximize the impulse given by (11) which is equivalent to maximizing the projected momentum  $P(t) \in \mathbb{R}$  of the hammer tip along the normal  $\mathbf{n}$  [15] defined as :

$$P(t) \triangleq m_{e,n}(\mathbf{q}) \dot{\mathbf{x}}_\nu^T \mathbf{n} \quad (12)$$

Then, from (11) and (12), we have :

$$i = \Delta P = P(t_c + \delta t) - P(t_c) \quad (13)$$

For a fixed given target normal velocity  $\dot{\mathbf{x}}_\nu^T \mathbf{n}$ , we see that in order to maximize the projected momentum of the hammer tip, we have to maximize the EMR, which is explained in Section IV.

### IV. OPTIMIZATION OF THE EFFECTIVE MASS USING QP

In this section, we will describe how to maximize the EMR using Quadratic Programming (QP).

#### A. Tasks

Let us first introduce what we call a "task". A task is an objective that the robot has to accomplish, for instance maintaining a posture or moving its end-effector to a desired position. To realize a task, we compute a desired configuration acceleration  $\ddot{\mathbf{q}}_d$ , then we track how well the robot realizes this task using a cost function expressed as a function of the actual configuration acceleration  $\ddot{\mathbf{q}}$ .

#### B. Formulation of the problem to solve

For this part, our decision variable is the configuration acceleration  $\ddot{\mathbf{q}}$ . In order to make it appear on the expression of the effective mass, we differentiate the EMR twice with respect to time :

$$\dot{m}_{e,n}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{dm_{e,n}}{d\mathbf{q}} \frac{d\mathbf{q}}{dt} = \nabla m_{e,n}^T(\mathbf{q}) \dot{\mathbf{q}} \quad (14)$$

$$\ddot{m}_{e,n}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = \frac{d}{dt} \{ \nabla m_{e,n}^T(\mathbf{q}) \} \dot{\mathbf{q}} + \nabla m_{e,n}^T(\mathbf{q}) \ddot{\mathbf{q}} \quad (15)$$

From (15), we can reconstruct  $m_{e,n}$  by discrete time integration. Let  $\Delta t$  be the timestep of the controller, then we have :

$$m_{e,n}(\mathbf{q}(t + \Delta t)) = m_{e,n} + \dot{m}_{e,n} \Delta t + \ddot{m}_{e,n} \frac{\Delta t^2}{2} \quad (16)$$

Taking  $\arg \max_{\ddot{\mathbf{q}}}$  on both sides, the two first terms of the right hand side of the equation are discarded because they do not depend on  $\ddot{\mathbf{q}}$  by (6) and (14) :

$$\arg \max_{\ddot{\mathbf{q}}} m_{e,n}(\mathbf{q}(t + \Delta t)) = \arg \max_{\ddot{\mathbf{q}}} \ddot{m}_{e,n} \quad (17)$$

Thus, in order to maximize  $m_{e,n}$  with respect to  $\ddot{\mathbf{q}}$ , we have to maximize  $\ddot{m}_{e,n}$ . Therefore, we want to solve :

$$\arg \max_{\ddot{\mathbf{q}}} \left\{ \frac{d}{dt} \{ \nabla m_{e,n}^T(\mathbf{q}) \} \dot{\mathbf{q}} + \nabla m_{e,n}^T(\mathbf{q}) \ddot{\mathbf{q}} \right\} \quad (18)$$

Again, because our decision variable is  $\ddot{\mathbf{q}}$ , we can drop the first term which does not depend on the configuration acceleration  $\ddot{\mathbf{q}}$  and we are left with :

$$\arg \max_{\ddot{\mathbf{q}}} \nabla m_{e,n}^T(\mathbf{q}) \ddot{\mathbf{q}} \quad (19)$$

which saves us the computation of the time derivative of the gradient.

As the QP is defined as a minimization problem, we can rewrite (19) as such in the following way:

$$\arg \min_{\ddot{\mathbf{q}}} -\nabla m_{e,n}^T(\mathbf{q}) \ddot{\mathbf{q}} \quad (20)$$

Let's call this objective function the Effective Mass Maximization Task (EMMT). The goal of this task is to increase as much as possible the EMR at the next iteration of the controller without its value dropping down. However, we cannot assert that the result will be a global maximum because our problem is not convex. If we want to try to find a global maximum, we might need to use other methods, although these ones are generally much slower.

#### C. Linear QP constraints to be respected

Our linear constraints for the QP solver are described as i) joint limits, ii) joint velocity limits, iii) joint acceleration limits, iv) joint torque limits, and v) dynamic constraints :

$$\mathbf{q}_{LB} \preceq \mathbf{q} \preceq \mathbf{q}_{UB} \quad (21)$$

$$\dot{\mathbf{q}}_{LB} \preceq \dot{\mathbf{q}} \preceq \dot{\mathbf{q}}_{UB} \quad (22)$$

$$\ddot{\mathbf{q}}_{LB} \preceq \ddot{\mathbf{q}} \preceq \ddot{\mathbf{q}}_{UB} \quad (23)$$

$$\boldsymbol{\tau}_{LB} \preceq \boldsymbol{\tau} \preceq \boldsymbol{\tau}_{UB} \quad (24)$$

$$\boldsymbol{\kappa}_\xi = (d_i, d_s, \xi_{offset})^T \quad (25)$$

where the subscripts **LB** and **UB** mean "Lower Bound" and "Upper Bound", respectively,  $\boldsymbol{\tau} \in \mathbb{R}^n$  is the actual vector of joint torques and the  $\preceq$  operator is the term-by-term comparison operator between two vectors of  $\mathbb{R}^n$ . For the dynamic constraints (25),  $d_i$  is the interaction distance,  $d_s$  is the security distance and  $\xi_{offset}$  is a fixed offset, all developed in Vaillant et al. [16].

#### D. Implementation of the EMMT

The posture task is a task where the robot has to maintain a given configuration  $\mathbf{q}$ , the paper from Cisneros et al. [17] gives a deeper look into it. From the complete cost function in our QP formulation :

$$\arg \min_{\ddot{\mathbf{q}}} \frac{1}{2} W_p \|\ddot{\mathbf{q}} + \mathbf{b}\|^2 - W_m \nabla m_{e,n}^T(\mathbf{q}) \ddot{\mathbf{q}} + A(\ddot{\mathbf{q}}) \quad (26)$$

the first term represents the posture task, the second term is the EMMT and  $A(\ddot{\mathbf{q}}) \in \mathbb{R}$  is the cost function associated to the other possible tasks that the robot has to realize (for instance a trajectory task).  $W_p, W_m \in \mathbb{R}^+$  are the relative weights associated to the posture task and the EMMT, respectively, and  $\mathbf{b} \in \mathbb{R}^n$  is a PD controller written as:

$$\mathbf{b} = K_p(\mathbf{q} - \mathbf{q}_d) + K_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d) - \ddot{\mathbf{q}}_d \quad (27)$$

where  $K_p$  and  $K_d$  are the proportional (stiffness) and derivative (damping) gains respectively,  $\mathbf{q}_d, \dot{\mathbf{q}}_d$  and  $\ddot{\mathbf{q}}_d \in \mathbb{R}^n$  are the desired configuration, desired configuration velocity and desired configuration acceleration, respectively.

The relationship between  $K_p$  and  $K_d$  is given by :

$$K_d = 2\sqrt{K_p} \quad (28)$$

To implement our EMMT task, we will modify (26) in the following way :

$$\arg \min_{\ddot{\mathbf{q}}} \frac{1}{2} W_p \|\ddot{\mathbf{q}} + \mathbf{b}\|^2 - \frac{W_m}{W_p^2} \nabla m_{e,n}(\mathbf{q}) \|\ddot{\mathbf{q}}\|^2 + A(\ddot{\mathbf{q}}) \quad (29)$$

subject to (21), (22), (23), (24) and (25).

This approach allows us to maximize the EMR online instead of optimizing offline. Using an offline method implies knowing the pose of the hitting target in advance everytime, and this would not work if we were to change the pose of the nail.

#### V. TRAJECTORY OF THE END-EFFECTOR

In this section, we present how the end-effector of the robot reaches the nail with a given velocity.

##### A. B-Spline trajectory task

The primary task of the end-effector is to reach the nail with the given velocity. To do that, we use a continuous Bézier curve as our trajectory, which is a B-Spline (the B-Spline is a generalization of the Bézier curve). Let us call that task the B-Spline trajectory task (BSTT).

A  $(N+1)$ -points Bézier curve  $\Gamma: \mathbb{R} \rightarrow \mathbb{R}^3$ , or  $N$ -th order Bézier curve, is defined as:

$$\Gamma(t) \triangleq \sum_{k=0}^N B_k^N \left( \frac{t-t_i}{T} \right) \mathbf{p}_k \quad \forall t \in [t_i, t_c] \quad (30)$$

where  $[t_i, t_c]$  is the time interval on which  $\Gamma$  is defined,  $t_c > t_i$ ,  $\mathbf{p}_k \in \mathbb{R}^3$  is the constant  $k$ -th control point of the curve,  $T = t_c - t_i$  and  $B_k^N$  is a Bernstein polynomial that acts as a weight [18], such that:

$$B_k^N \left( \frac{t-t_i}{T} \right) \triangleq \binom{N}{k} \left( \frac{t-t_i}{T} \right)^k \left( 1 - \frac{t-t_i}{T} \right)^{N-k} \quad (31)$$

The first control point of the curve,  $\mathbf{p}_0$ , is the 3D initial position of the hammer tip,  $\mathbf{x}_{\nu}^{t_i} \in \mathbb{R}^3$  (the subscript "i" stands for "initial"), and the last control point,  $\mathbf{p}_N$ , is the 3D position of the nail  $\mathbf{c}^{nail} \in \mathbb{R}^3$ , both with respect to the fixed world frame. Thus :

$$\mathbf{p}_0 = \mathbf{x}_{\nu}^{t_i} \quad (32)$$

$$\mathbf{p}_N = \mathbf{c}^{nail} \quad (33)$$

These points have the property as acting as waypoints for the trajectory, which is not the case for the potential in-between points.

##### B. Curve constraints

To make the end-effector reach the desired velocity when hitting the nail, we constrain the curve by adding four control points  $\mathbf{p}_{c1}, \mathbf{p}_{c2}, \mathbf{p}_{cN-1}$  and  $\mathbf{p}_{cN-2} \in \mathbb{R}^3$ . These added points count towards the degree of the curve. The expression of the added points is given by [19]:

$$\mathbf{p}_{c1} = \mathbf{p}_0 + \frac{T}{N} \boldsymbol{\kappa}_{\dot{\mathbf{x}}_{\nu}^{t_i}} \quad (34)$$

$$\mathbf{p}_{c2} = 2\mathbf{p}_{c1} - \mathbf{p}_0 + \frac{T^2}{(N-1)(N-2)} \boldsymbol{\kappa}_{\ddot{\mathbf{x}}_{\nu}^{t_i}} \quad (35)$$

which are added just after  $\mathbf{p}_0$ , where  $\boldsymbol{\kappa}_{\dot{\mathbf{x}}_{\nu}^{t_i}} \in \mathbb{R}^3$  are the initial linear velocity constraints and  $\boldsymbol{\kappa}_{\ddot{\mathbf{x}}_{\nu}^{t_i}} \in \mathbb{R}^3$  are the initial linear acceleration constraints. Also :

$$\mathbf{p}_{cN-1} = \mathbf{p}_N - \frac{T}{N-1} \boldsymbol{\kappa}_{\dot{\mathbf{x}}_{\nu}^{t_c}} \quad (36)$$

$$\mathbf{p}_{cN-2} = 2\mathbf{p}_{cN-1} - \mathbf{p}_N + \frac{T^2}{(N-1)(N-2)} \boldsymbol{\kappa}_{\ddot{\mathbf{x}}_{\nu}^{t_c}} \quad (37)$$

where  $\boldsymbol{\kappa}_{\dot{\mathbf{x}}_{\nu}^{t_c}} \in \mathbb{R}^3$  are the linear final velocity constraints and  $\boldsymbol{\kappa}_{\ddot{\mathbf{x}}_{\nu}^{t_c}} \in \mathbb{R}^3$  are the final linear acceleration constraints. Thus, if we add the curve constraints, we will have six control points in total, hence a 5<sup>th</sup> order Bézier curve.

##### C. Hammer tip velocity and acceleration

The desired velocity  $\dot{\mathbf{x}}_{\nu,d}$  and acceleration  $\ddot{\mathbf{x}}_{\nu,d}$  of the hammer tip from the initial instant  $t_i$  to the instant of contact  $t_c$  expressed with respect to the world frame  $w$  are given by the first and second time derivative of (30), respectively :

$$\dot{\mathbf{x}}_{\nu,d}(t) = \dot{\Gamma}(t) \quad \forall t \in [t_i, t_c] \quad (38)$$

$$\ddot{\mathbf{x}}_{\nu,d}(t) = \ddot{\Gamma}(t) \quad \forall t \in [t_i, t_c] \quad (39)$$

Both the velocity and acceleration of the hammer tip are affected by the curve constraints (more precisely, the added control points) defined in Section V-B.

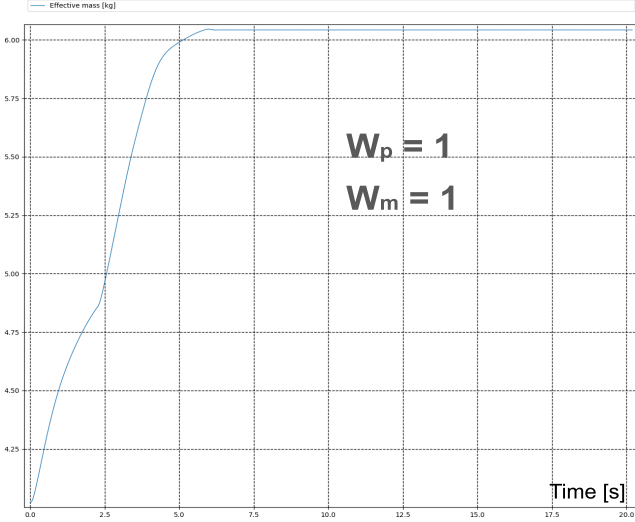


Fig. 2. Time evolution of the EMR with only a posture task active

#### D. Orientation of the hammer tip

In addition to reaching the nail, we have to orient the hammer tip such that the impact with the nail axis is *collinear*. Let  $\mathbf{G}^h, \mathbf{G}^{nail} \in \mathbb{R}^3$  be the center of mass of the hammer tip and the center of mass of the nail, respectively. A collinear impact is an impact where  $\mathbf{G}^h, \mathbf{G}^{nail}$  and the contact points  $\mathbf{x}_\nu$  and  $\mathbf{c}^{nail}$  are aligned [14], see Fig. 1. We constraint two rotation DoFs: the roll and the pitch of the hammer tip, and we let the yaw be free. Indeed, we want to align three vectors: the vector normal to the contact surface of the hammer tip which we will call  $\mathbf{n}^h \in \mathbb{R}^3$  (expressed in the hammer tip frame), the  $\mathbf{n}$  vector and finally the vector between the hammer tip and the contact point of the nail. The latter is already done by the path to the nail.

Let  $\mathbf{R}_{w \rightarrow h}$  be the rotation matrix describing the rotation of the hammer tip from the world frame,  $\epsilon = \mathbf{n}^h - \mathbf{n}$  the error vector between the desired vector ( $\mathbf{n}$ ) and the actual vector ( $\mathbf{n}^h$ ),  $\dot{\mathbf{x}}_\omega \in \mathbb{R}^3$  the angular velocity vector of the hammer tip in the world frame given by :

$$\dot{\mathbf{x}}_\omega = -\mathbf{R}_{w \rightarrow h} [\mathbf{n}^h]_\times \mathbf{J}_\omega \dot{\mathbf{q}} \quad (40)$$

where  $[\cdot]_\times : \mathbb{R}^3 \rightarrow \mathbb{R}^{3 \times 3}$  is the skew-symmetric operator defined, for all  $\mathbf{a} = (a_x, a_y, a_z)^T \in \mathbb{R}^3$ , as :

$$[\mathbf{a}]_\times = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \quad (41)$$

The time derivative of (40) is :

$$\ddot{\mathbf{x}}_\omega = -\dot{\mathbf{R}}_{w \rightarrow h} [\mathbf{n}^h]_\times \mathbf{J}_\omega \dot{\mathbf{q}} - \mathbf{R}_{w \rightarrow h} [\dot{\mathbf{n}}^h]_\times (\mathbf{J}_\omega \ddot{\mathbf{q}} + \dot{\mathbf{J}}_\omega \dot{\mathbf{q}}) \quad (42)$$

Let us call this task the Vector Orientation Task (VOT).

The cost function of the task is :

$$A_{VOT}(\ddot{\mathbf{q}}) = \|K_{p,VOT}\epsilon + K_{d,VOT}\dot{\epsilon} - \ddot{\mathbf{x}}_\omega(\ddot{\mathbf{q}})\|^2 \quad (43)$$

where  $K_{p,VOT}, K_{d,VOT} \in \mathbb{R}^+$  are its stiffness and damping, respectively.

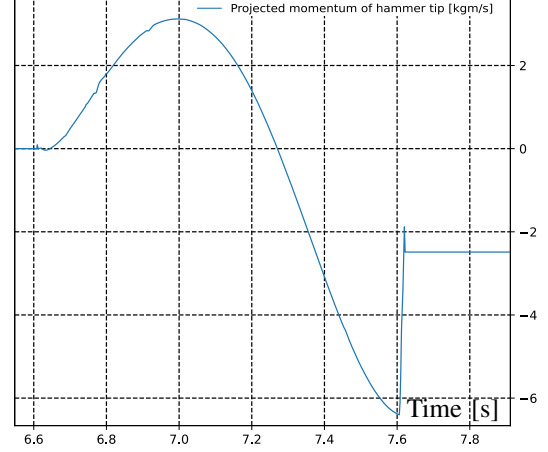


Fig. 3. Time evolution of the projected momentum along the normal  $\mathbf{n}$

## VI. SIMULATIONS WITH THE ROBOT

In this section, we present the simulations made on our humanoid robot HRP-5P. The setting is a fixed hammer in one of the robots' grippers (the left one in our case), a fully known position and orientation of the nail and a prefixed desired hitting velocity of the hammer tip on the nail. Simulations were done using `mc_mujoco` and `mc_rtc` [20] in torque control mode with a timestep  $\Delta t = 0.002s$ . The robot is equipped with a hammer in its left gripper and its floating base is fixed. The hammer used weighs 0.792 kg, has a length of 328 mm and its contact surface is circular with radius 15.5 mm. The nail is fixed and "floating" in the environment because of the current framework limitations, and has a length of 88.03 mm and its head has a radius of 6.95 mm. The tasks at play are the EMMT within the posture task defined in Section IV-B, the BSTT defined in Section V-A and the VOT defined in Section V-D. The gradient of the effective mass  $\nabla m_{e,n}$  is computed by backward difference. In (27), we set  $K_p = 5$ , thus by (28) we have  $K_d = 4.47$ . The entire simulation goes as follows: i) the robot starts at its half-sitting posture ii)  $t_i$  is recorded and the robot starts the hammering motion until an impact is detected on the nail, then  $t_c$  is recorded at the same time, and finally iii) the robot goes back to its half-sitting posture. An impact is detected on the nail thanks to a nail sensor plugin written in ROS2 and added to `mc_rtc`: there is an impact if the absolute value of one coordinate of the 3D force applied to the nail is at least greater than some threshold  $f_e$  which has been chosen to be 1N to cut off the very small "offset" forces of the simulator, as the magnitude of an impulse is much higher than this value.

We first assess the effectiveness of the EMMT (Section IV) in Fig. 2 with  $W_m = 1$  and  $W_p = 1$  by isolating the augmented posture task (with EMMT), i.e. we only have the posture task and EMMT active and no other task is active. We see that the EMR is growing over time until it converges to a maximum value. As expected from (29), increasing  $W_m$  induces a larger joint acceleration, thus the EMR converges faster and to a larger value, however it can lead to stability

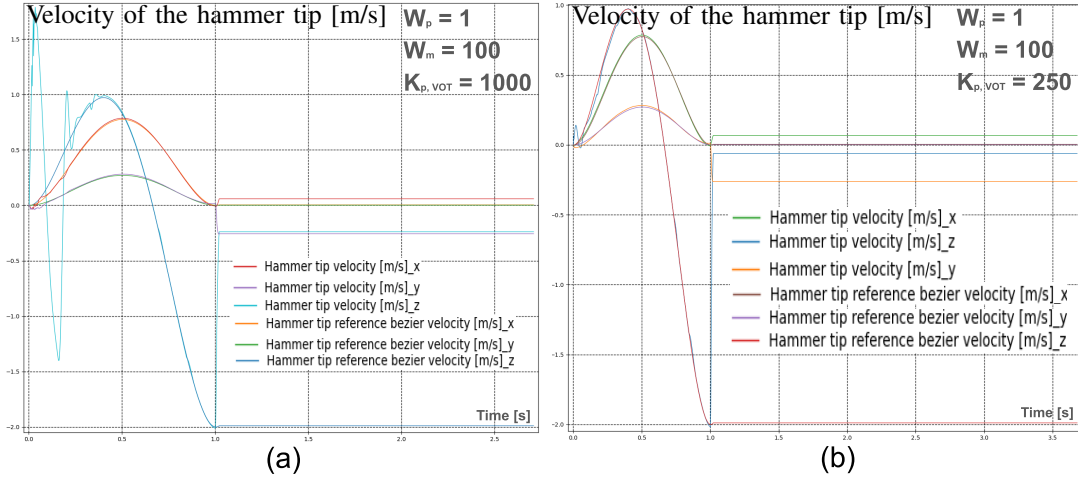


Fig. 4. Velocity tracking of the hammer tip for different  $K_{p,VOT}$  values

issues with the robot.

Then, we use BSTT and VOT, which are already implemented in `mc_rtc` and we display the tracking of the theoretical trajectory given by the Bézier curve (30), with the following curve constraints:  $\kappa_{\dot{\mathbf{x}}_\nu^{t_i}} = (0, 0, 0)^T$ ,  $\kappa_{\dot{\mathbf{x}}_\nu^{t_c}} = (0, 0, 0)^T$ ,  $\kappa_{\dot{\mathbf{x}}_\nu^{t_c}} = (0, 0, 0)^T$  and especially

$$\kappa_{\dot{\mathbf{x}}_\nu^{t_c}} = \mathbf{R}_{w \rightarrow \text{nail}}^T \text{nail} \dot{\mathbf{x}}_\nu^{t_c} \quad (44)$$

where  $\text{nail} \dot{\mathbf{x}}_\nu^{t_c} \in \mathbb{R}^3$  is the input velocity of the hammer tip at the instant of contact  $t_c$  expressed in the nail frame. The nail is placed at  $\mathbf{c}^{\text{nail}} = (0.75, 0.40, 0.90)^T$  from the origin of  $w$  with the distances given in meters, and oriented as depicted by Fig. 1, i.e perpendicular to an horizontal plane. We set  $T = 1\text{s}$ ,  $W_{BSTT} = 100$ ,  $W_{VOT} = 10$ ,  $W_m = 100$ ,  $W_p = 1$ ,  $\text{nail} \dot{\mathbf{x}}_\nu^{t_c} = (0, 0, -2.0)^T$  m/s, the stiffness of the BSTT to  $K_{p,BSTT} = 10000$  and the stiffness of the VOT to  $K_{p,VOT} = 1000$  with their damping given by (28). We see the tracking of the hammer tip velocity in Fig. 4 (a). We remark that the hammer tip properly reaches  $\text{nail} \dot{\mathbf{x}}_\nu^{t_c}$  until the sudden spike in velocity at the end, which is due to a collision with the nail. However, we also see some unwanted oscillations on the  $z$  coordinate: this is due to the high stiffness (1000) of the VOT. The final orientation error has been recorded to be 4.16 deg. We decrease the stiffness of the VOT to 250 in Fig. 4 (b) and we record an orientation error of 11.88 deg. Increasing  $K_{p,VOT}$  decreases the error between the desired and actual final hammer tip orientations at the cost of larger oscillations at the beginning. The projected momentum of the hammer tip is computed using (12) and is shown in Fig. 3.

## VII. CONCLUSION

In this work, we presented a way to implement a hammering task on a humanoid robot. Then, we proposed our method to maximize the EMR using QP and its gradient and we suggested a trajectory constructed from a constrained 5<sup>th</sup> order Bézier curve. Simulations showed that the EMR is getting maximized and the trajectory given by the Bézier curve is properly followed by the end-effector. Possible improvements include an estimation of the nail penetration in the wood along with a simulation of the dynamics of the nail, a way to build our path and orientation to the nail by efficiently considering collision constraints with the environment or a post-impact that utilizes the post-impact

velocity of the hammer tip to prepare for the next hit, as the robot only goes back to its initial posture for now. In the future, we want to work on the stability problem of the robot potentially induced by the post-impact rebound of the hammer, minimizing the damage that the robot must endure on its joints during the hit to avoid damage as much as possible and detection of the nail via computer vision.

## REFERENCES

- [1] Scott P. Schneider. "Musculoskeletal Injuries in Construction: A Review of the Literature". In: *Applied Occupational and Environmental Hygiene* 16.11 (Nov. 2001), pp. 1056–1064.
- [2] Julie N. Côté et al. "Differences in Multi-Joint Kinematic Patterns of Repetitive Hammering in Healthy, Fatigued and Shoulder-Injured Individuals". In: *Clinical Biomechanics* 20.6 (July 2005), pp. 581–590.
- [3] Nilanthy Balendra and Joseph E. Langenderfer. "Effect of Hammer Mass on Upper Extremity Joint Moments". In: *Applied Ergonomics* 60 (Apr. 2017), pp. 231–239.
- [4] Tadej Petrič et al. "Hammering Does Not Fit Fitts' Law". In: *Frontiers in Computational Neuroscience* 11 (May 2017).
- [5] F. Aghazadeh and A. Mital. "Injuries Due to Handtools: Results of a Questionnaire". In: *Applied Ergonomics* 18.4 (Dec. 1987), pp. 273–278.
- [6] Yuan-Ho Chen and Yi-Lang Chen. "An Optimal Effective Working Height for Hammering Tasks". In: *Work* 61.2 (Nov. 2018), pp. 181–187.
- [7] Kenji Kaneko et al. "Humanoid Robot HRP-5P: An Electrically Actuated Humanoid Robot With High-Power and Wide-Range Joints". In: *IEEE Robotics and Automation Letters* 4.2 (Apr. 2019), pp. 1431–1438.
- [8] Rafael Cisneros, Kazuhito Yokoi, and Eiichi Yoshida. "Impulsive Pedipulation of a Spherical Object with 3D Goal Position by a Humanoid Robot: A 3D Targeted Kicking Motion Generator". In: *International Journal of Humanoid Robotics* 13.02 (June 2016), p. 1650003. (Visited on 11/01/2025).
- [9] Jari J. van Steen, Nathan van de Wouw, and Alessandro Saccon. "Robot Control for Simultaneous Impact Tasks via Quadratic Programming-based Reference Spreading". In: *2022 American Control Conference (ACC)*. June 2022, pp. 3865–3872. arXiv: 2111.05211 [cs].
- [10] Francesco Tassi et al. "IMA-catcher: An Impact-aware Nonprehensile Catching Framework Based on Combined Optimization and Learning". In: *The International Journal of Robotics Research* (June 2025), p. 02783649251345851.
- [11] Yuquan Wang and Abderrahmane Kheddar. "Impact-Friendly Robust Control Design with Task-Space Quadratic Optimization". In: *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019.
- [12] The `mc_rtc` framework. URL: [https://jrl.cnrs.fr/mc\\_rtc/](https://jrl.cnrs.fr/mc_rtc/).
- [13] Magnus Bjerkeng and Kristin Y. Pettersen. "A New Coriolis Matrix Factorization". In: *2012 IEEE International Conference on Robotics and Automation*. May 2012, pp. 4974–4979.
- [14] W. J. Stronge. *Impact Mechanics*. 2nd ed. Cambridge University Press, 2018.
- [15] Luciano da Fontoura Costa. *Single Point Particle Motion: Mass, Force, Momenta, Impulse, Work*. 2022. {hal-03907279}.
- [16] Joris Vaillant et al. "Multi-Contact Vertical Ladder Climbing with an HRP-2 Humanoid". In: *Autonomous Robots* 40.3 (Mar. 2016), pp. 561–580.
- [17] Rafael Cisneros et al. "Robust Humanoid Control Using a QP Solver with Integral Gains". In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Madrid: IEEE, Oct. 2018, pp. 7472–7479.
- [18] David Salomon. *Computer Graphics & Geometric Modeling*.
- [19] The `ndcurves` library. URL: <https://github.com/loco-3d/ndcurves/blob/devel/doc/curves.pdf>.
- [20] Rohan P. Singh, Pierre Gergondet, and Fumio Kanehiro. *Mc-Mujoco: Simulating Articulated Robots with FSM Controllers in MuJoCo*. Sept. 2022. arXiv: 2209.00274 [cs].